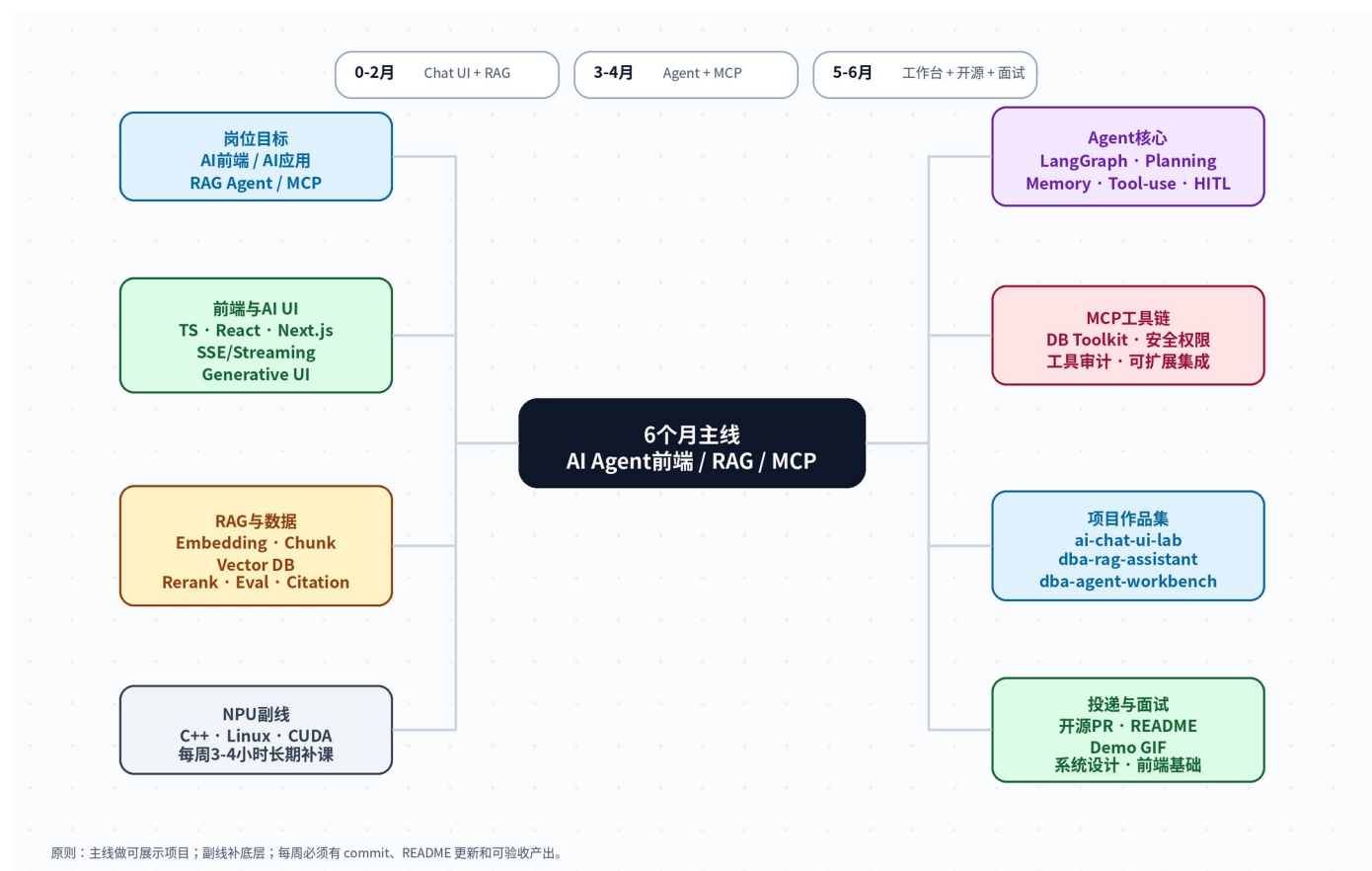


AI Agent 前端 / RAG / MCP

6 个月详细学习计划

基于阿里云 AI 前端 JD + 字节 NPU JD 的路线设计

主线：AI Agent 前端 / AI 应用工程 | 副线：NPU 系统软件长期补课



版本：2026-07-01 | 建议周期：24 周 | 适合节奏：工作日 2-3 小时 + 周末 6-8 小时

目录

1. 学习定位：主线冲 AI Agent 前端，副线保留 NPU 长期补课
2. 岗位能力拆解与学习优先级
3. 6 个月最终产出与 GitHub 作品集
4. 24 周详细学习计划
5. 每日/每周执行节奏
6. 核心项目设计：DBA Agent Workbench
7. Mac mini / 电脑开发环境清单
8. NPU 副线学习计划
9. 面试准备与简历包装
10. 参考资料

1. 学习定位

结论：短期主线不要走 NPU 系统软件，也不要从零训练大模型；你最该冲的是 AI Agent 前端 / AI 应用工程 / RAG / MCP。原因是这条路线更接近阿里云 JD，也能复用你的 DBA/数据库经验，6 个月内更容易做出能展示、能投递的作品。

路线	目标岗位	6 个月目标	投入比例
主线	AI 前端工程师 / AI 应用工程师 / Agent 工程师 / RAG 工程师	完成 4-5 个项目：Chat UI、RAG、Agent、MCP、Workbench、组件库	80%-85%
副线	NPU 系统软件 / GPU Runtime 长期方向	每周 3-4 小时补 C++、Linux、CUDA、Runtime；暂不作为短期投递主战场	15%-20%

主线关键词：TypeScript、React、Next.js、SSE/Streaming、Vercel AI SDK、RAG、LangGraph、MCP、Function Calling、Human-in-the-loop、AI-Native UI、Docker、CI/CD、Observability。

副线关键词：C++17/20、Linux 系统编程、CMake、gdb、CUDA、ROCm、GPU memory hierarchy、Kernel、Runtime、SDK、Driver、Firmware。

2. 岗位能力拆解与优先级

JD 能力	你要掌握到什么程度	对应项目	优先级
TypeScript / React / 现代前端	能做复杂状态、组件抽象、性能优化、移动端适配	ai-chat-ui-lab / dba-agent-workbench	P0
流式对话与 AI UI	能处理 SSE、停止生成、错误恢复、工具调用状态展示	ai-chat-ui-lab	P0
RAG	懂 chunk、embedding、retriever、rerank、citation、eval	dba-rag-assistant	P0
Agent 编排	能用 LangGraph 设计 planner/executor/analyser/finalizer	db-troubleshooting-agent	P0
MCP	能写 MCP Server，把数据库工具标准化暴露给 Agent	mcp-db-toolkit	P0
Generative UI	能让模型输出结构化 UI，并安全渲染 artifact	dba-agent-workbench	P1
组件库 / Design System	能抽出 ChatMessage、ToolCallCard、Timeline、ArtifactPanel	ai-agent-ui-components	P1
部署与可观测性	能 Docker Compose 一键部署，并记录 trace/latency/success rate	agent-eval-harness	P1
NPU/GPU 系统软件	先补基础：C++、Linux、CUDA，不抢主线时间	cuda-npu-lab	P2

3. 6 个月最终产出

6 个月结束时，不是“学完一堆课”，而是要有可以投简历的作品集。下面是最低交付标准。

仓库	定位	核心功能	简历关键词	验收
ai-chat-ui-lab	AI 聊天前端基础	会话、流式输出、停止生成、重试、Markdown/代码块、工具面板	React、TS、SSE、AI SDK UI	有 GIF、有 README、有移动端适配
dba-rag-assistant	数据库知识库问答	文档上传、切块、向量检索、引用回答、bad case	RAG、Embedding、Vector DB、Eval	30+ 测试问题，回答带来源
mcp-db-toolkit	MCP 数据库工具链	健康检查、慢 SQL 分析、	MCP、Tool Calling、权限	MCP Inspector 可调通

		SQL Review、知识库搜索	审计	
dba-agent-workbench	核心作品：AI Agent 工作台	对话、执行轨迹、Human Approval、Artifact、报告导出	LangGraph、Agent、AI-Native UI	可完整演示一个数据库故障排查流程
ai-agent-ui-components	AI 产品组件库	ChatMessage、ToolCallCard、Timeline、ArtifactPanel、主题	Design System、组件抽象	独立 package 和示例页面
agent-eval-harness	Agent 评测与观测	DBA 测试集、工具成功率、延迟、引用命中率、失败样例	Observability、Eval、Trace	50+ case，输出评测报告

4. 24 周详细学习计划

执行原则：每周有明确交付物；每周至少 5 次 commit；周末必须更新 README、截图/GIF 和学习日志。

第 1-6 周：AI 前端 + RAG 基础

周	主题	学习任务	实战任务	验收标准
1	环境与前端骨架	搭好 Mac mini/电脑环境；复习 TS 类型、React Hooks、Next.js App Router；明确项目 monorepo 结构。	创建 ai-chat-ui-lab：会话列表、聊天窗口、右侧工具面板；不接模型，先 fake 数据。	GitHub 初始化；README 有架构图草稿；页面在 1440px 和移动端都可用。
2	Streaming Chat UI	学习 SSE、WebSocket、AbortController、消息状态机、Markdown/代码块渲染。	接 fake streaming API：逐字输出、停止生成、重新生成、错误重试、复制代码。	录 GIF；能解释 SSE 与 WebSocket 区别；写一篇流式渲染笔记。
3	LLM Gateway	学习 OpenAI-compatible API、模型切换、超时、重试、token/耗时统计。	创建 llm-gateway：统一封装模型调用，支持 streaming、日志、fallback。	前端接真实模型；每次请求记录 model、latency、tokens、error。
4	RAG 入门	学习 Embedding、Chunk、Vector DB、topK、metadata、citation。	创建 dba-rag-assistant：上传 DBA 文档、切块、入库、提问、返回引用。	完成 10 个 DBA 问题测试；能说清 RAG 数据流。
5	RAG 质量优化	学习 query rewrite、multi-query、rerank、guardrail、bad case 记录。	增加命中文档展示、相似度分数、回答引用、找不到则拒答。	沉淀 30 条测试集；README 写清 chunk 策略和评测结果。
6	Function Calling	学习 tool schema、参数校验、工具选择、工具结果回填、危险操作确认。	为 DBA Agent 增加 search_docs、explain_sql、analyze_slow_query、diagnose_deadlock 等工具。	所有工具有 zod/pydantic schema；危险动作默认只读或需要确认。

第 7-12 周：Agent + MCP + 工作台雏形

周	主题	学习任务	实战任务	验收标准
7	LangGraph 基础	学习 StateGraph、Node、Edge、checkpoint、streaming、human-in-the-loop。	实现 planner -> retriever -> tool_executor -> analyzer -> finalizer 的最小图。	能在控制台看到每个节点状态；失败后可重试。
8	数据库排障 Agent	把真实 DBA 排障流程结构化：CPU 飙高、慢 SQL、死锁、主从延迟。	创建 db-troubleshooting-agent：输入故障，输出 checklist、诊断 SQL、风险提示。	完成 4 类故障流程；每类至少 5 个测试问题。
9	MCP Server	学习 MCP tools/resources/prompts、server/client、权限、审计。	创建 mcp-db-toolkit：暴露数据库健康检查、知识库搜索、SQL Review 等工具。	本地 MCP Inspector 可调通；每次工具调用有日志。
10	MCP + Agent 集成	理解 MCP 与普通 REST API 的差异：标准化工具发现、上下文接入、客户端复用。	让 Agent 通过 MCP 调用 DB Toolkit；前端展示工具 schema 和结果。	能录制“Agent 调用 MCP 工具链”的完整演示。
11	Agent Workbench	学习 AI 产品前端：执行轨迹、任务状态、引用文档、artifact 面板。	创建 dba-agent-workbench：左会话、中对话、右执行轨迹。	支持 planning/retrieving/calling/waiting/analyzing/done 状态展示。
12	Human Approval 与审计	学习生产安全：权限分级、敏感信息过滤、操作确认、审计回放。	增加工具调用审批、拒绝/继续、审计日志、执行回放。	能解释 prompt injection 与 tool abuse 的防护思路。

第 13-18 周：Generative UI + 部署 + 开源

周	主题	学习任务	实战任务	验收标准
13	Generative UI	学习 JSON schema 输出、Artifact UI、动态组件、安全渲染。	让 Agent 生成慢 SQL 分析卡片、执行计划表、恢复时间线、风险 Badge。	组件可编辑、可复制、可导出；异常 JSON 不崩页面。
14	Design System / 组件库	学习组件抽象、主题、响应式、可访问性、Storybook 可选。	抽出 ai-agent-ui-components：ChatMessage、ToolCallCard、AgentStepTimeline、ArtifactPanel。	组件有独立 README 和示例；主项目通过 package 引用。
15	前端性能与稳定性	学习虚拟列表、token 合并刷新、memo、错误边界、断线重连。	优化长消息和 1000 条消息场景；工具失败可重试。	写性能报告；Playwright 覆盖核心链路。
16	Docker 私有化部署	学习 Dockerfile、Compose、Postgres/pgvector、Redis、Nginx、环境变量。	一条 docker compose up 启动 web、api、agent-worker、postgres、redis、mcp-server。	README 提供本地/服务器部署步骤；能在新环境跑通。
17	可观测性与 Eval	学习 trace、metrics、latency、success rate、成本统计、评测集。	创建 agent-eval-harness：50 条 DBA 场景评测、工具成功率、引用命中率。	输出 eval 报告；失败样例进入 bad_cases.md。
18	开源贡献	选择 Dify、LangGraph、Open WebUI、Firecrawl、MCP servers 中一个做小贡献。	提交 1 个文档/示例/bug 复现 PR；同时提 2 个高质量 issue。	至少有 1 个 merged PR 或被维护者有效回复。

第 19-24 周：作品集 + 面试 + 投递

周	主题	学习任务	实战任务	验收标准
19	作品集打磨	学习项目包装：截图、GIF、架构图、技术难点、演示脚本。	完善 3 个 pinned repos：workbench、mcp toolkit、ui components。	每个 README 都有背景、架构、启动、截图、难点、后续计划。
20	AI 前端面试	复习 React、TS、浏览器、网络、安全、性能、流式渲染。	整理 60 道问答；给项目补“面试可讲点”。	能 10 分钟讲清 dba-agent-workbench 架构。
21	Agent 系统设计	复习 RAG、Agent、MCP、权限、观测、降级、成本控制。	设计题：企业知识库 Agent / 数据库排障 Agent / AI 工具平台。	每个系统设计都有架构图、数据流、风险和扩展方案。
22	简历与投递	提炼关键词：AI 前端、RAG、Agent、MCP、AI-Native UI、Observability。	制作 1 页项目型简历；准备 Boss/拉勾/猎聘投递话术。	投递 20 个匹配岗位；记录 JD 关键词和反馈。
23	针对性补短板	根据面试反馈补 React 基础、系统设计、RAG 评测、MCP 安全。	给项目补缺：测试、错误处理、权限、部署文档。	完成第二轮 mock interview；形成错题本。
24	Release v1.0	整理 6 个月成果，做最终演示和复盘。	给 dba-agent-workbench 打 v1.0 tag，发布 demo 视频/GIF，写总结文章。	可直接投递；简历、作品集、演示脚本完整。

5. 每日 / 每周执行节奏

场景	时间安排	产出要求
工作日 2-3 小时	30 分钟官方文档 90 分钟写代码 30 分钟调试/测试 20 分钟学习日志 10 分钟 commit	每天至少 1 个小功能或 1 个 bugfix；commit message 写清楚做了什么。
周末 6-8 小时	3 小时项目功能 1.5 小时补文档/源码 1 小时 README/截图/GIF 1 小时复盘和下周计划	每周结束必须能演示一个可见变化。
每月复盘	项目可用性 简历关键词覆盖 面试题错题 开源参与记录	每月产出 1 篇技术文章或项目复盘。

每日学习日志模板：

```
# 2026-xx-xx

## 今天完成
-

## 遇到的问题
-

## 明天计划
-

## 面试可讲点
-
```

6. 核心项目设计：DBA Agent Workbench

这个项目是你的主作品。它要把你的数据库背景转化为 AI Agent 工程优势，而不是做一个普通聊天机器人。

推荐架构：

```
dba-agent-workbench/  
  apps/  
    web/           # Next.js + React + TypeScript  
    api/           # Node.js / Python API Gateway  
    agent-worker/  # LangGraph Agent Runtime  
  packages/  
    ui/            # AI Agent UI Components  
    mcp-db-toolkit/ # MCP Server: DB tools/resources/prompts  
    eval/          # Agent Eval Harness  
  infra/  
    docker-compose.yml  
    postgres-init.sql  
  docs/  
    architecture.md  
    eval-report.md  
    demo-script.md
```

模块	职责	面试可讲点
Web 前端	对话、流式输出、执行轨迹、工具审批、Artifact 面板、报告导出	复杂 AI UI 状态机、SSE 渲染性能、组件抽象
API Gateway	统一模型调用、认证、日志、限流、模型切换、错误重试	OpenAI-compatible API、token/latency/cost 统计
LangGraph Agent	Planner、Retriever、Tool Executor、Analyzer、Finalizer	状态图、Human-in-the-loop、持久化、失败恢复
MCP DB Toolkit	数据库健康检查、慢 SQL 分析、SQL Review、知识库检索	MCP Server、工具权限、审计、安全边界
RAG Knowledge Base	DBA 文档切块、向量索引、检索、rerank、引用来源	RAG 质量优化、评测集、bad case 分析
Eval/Observability	执行 trace、工具成功率、引用命中率、失败原因、回放	生产化思维、AI 应用可靠性

7. Mac mini / 电脑开发环境清单

按 Mac mini 准备环境；如果你不是 Mac，也可以按同样工具链迁移到 Linux。

- Homebrew、Git、GitHub CLI、VS Code/Cursor。
- Node.js LTS：建议用 nvm 管理；包管理器用 pnpm。
- Python 3.12+：建议用 uv 管理虚拟环境和依赖。
- Docker Desktop 或 OrbStack：用于 Postgres、Redis、Dify、Open WebUI、本地服务编排。
- PostgreSQL + pgvector：作为 RAG 向量库和业务数据存储。
- Redis：用于任务队列、缓存、Agent 状态临时存储。
- Ollama：用于本地模型验证；模型可从 Qwen/Llama 系列开始。
- 常用端口：3000 web、8000 api、5432 postgres、6379 redis、11434 ollama、9000 mcp。

建议第一天就跑通这个最小开发栈：

```
# 示例：项目依赖
node -v
pnpm -v
python --version
docker --version

# 示例：本地依赖
docker compose up -d postgres redis

# 示例：前端
pnpm install
pnpm dev
```

8. NPU 副线学习计划

NPU 岗位需要 C++、Linux、Firmware/Driver、CUDA/ROCm、体系结构、Runtime/SDK，这不是 2-3 个月能速成的方向。建议每周 3-4 小时长期补课，作为 12-18 个月的硬核备选路线。

月份	学习重点	实战产出	验收标准
第 1 月	C++17/20、CMake、gdb、RAII、智能指针、线程同步	cpp-runtime-lab：线程池、任务队列、内存池雏形	能讲清对象生命周期、锁、条件变量、死锁排查
第 2 月	Linux 系统编程：进程/线程、epoll、mmap、文件 IO、perf	mini-async-runtime：简单事件循环和异步任务	能用 strace/perf/gdb 定位问题
第 3 月	Linux Driver 基础：字符设备、ioctl、内核模块、用户态/内核态	toy-device-driver：可读写字符设备	能解释 ioctl、内核态/用户态拷贝风险
第 4 月	CUDA 基础：thread/block/grid、shared memory、kernel、stream	cuda-vector-add、reduce、matmul naive 版	能讲清 GPU 执行模型和显存层次
第 5 月	CUDA 优化：coalescing、tiling、occupancy、profiling	优化 matmul，并写性能报告	能用 Nsight/benchmark 对比优化前后
第 6 月	Runtime/SDK 思路：图执行、算子调度、内存复用、接口设计	mini-npu-runtime 模拟器	能讲清 Runtime 与 Driver/Firmware 的边界

9. 面试准备与简历包装

你最终要把自己包装成“能把 AI 接进真实业务系统的人”，而不是只会调 Prompt 的人。

AI 前端 / Agent 岗必须能回答：

- SSE 与 WebSocket 的区别，以及为什么 LLM token streaming 常用 SSE。
- 前端如何处理流式输出、停止生成、重试、断线恢复和长文本性能。
- Function Calling 的参数校验、工具失败恢复、危险工具审批怎么做。
- LangGraph 中 StateGraph、Node、Edge、checkpoint、human-in-the-loop 的作用。
- MCP Server 和普通 REST API 的区别；工具权限和审计怎么设计。
- RAG 为什么会答错；chunk、topK、rerank、citation、eval 怎么调。
- 如何防 Prompt Injection、Tool Poisoning、越权工具调用。
- AI-Native UI / Generative UI 如何安全渲染模型输出的结构化结果。
- 一个企业内部数据库排障 Agent 的系统设计：数据流、权限、观测、降级。

项目型简历描述模板：

基于 React/TypeScript/Next.js 构建 DBA Agent Workbench，实现对话式故障排查、流式输出、工具调用轨迹、人工确认和排障报告生成。

基于 LangGraph 设计 Planner/Executor/Analyzer/Finalizer 多节点 Agent 流程，支持 RAG 检索、MCP 工具调用、短期上下文和故障排查状态流转。

实现 MCP DB Toolkit，将数据库健康检查、慢 SQL 分析、执行计划解析、知识库检索等能力封装为可扩展工具，并接入 Agent 调用链路。

构建 Agent Eval Harness，沉淀 50+ DBA 场景测试集，统计回答引用命中率、工具调用成功率、平均延迟和失败原因。

10. 参考资料

优先看官方文档，不要先刷二手教程。下面这些是本计划用到的主参考。

- [LangGraph Overview](https://docs.langchain.com/oss/python/langgraph/overview): <https://docs.langchain.com/oss/python/langgraph/overview>
- [Model Context Protocol Intro](https://modelcontextprotocol.io/docs/getting-started/intro): <https://modelcontextprotocol.io/docs/getting-started/intro>
- [Vercel AI SDK Introduction](https://ai-sdk.dev/docs/introduction): <https://ai-sdk.dev/docs/introduction>
- [Dify Documentation](https://docs.dify.ai/): <https://docs.dify.ai/>
- [NVIDIA CUDA Programming Guide](https://docs.nvidia.com/cuda/cuda-programming-guide/index.html): <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>
- [AMD ROCm Documentation](https://rocm.docs.amd.com/en/latest/): <https://rocm.docs.amd.com/en/latest/>

最后提醒：每天最重要的是 commit。学习路线的价值不在收藏资料，而在 6 个月后能打开 GitHub，看到持续迭代的项目、截图、README、评测报告和开源记录。